

Detailný rozbor projektu — Spracovanie vstupných údajov v Kubernetes

exam-horvath

2026-06-06

0. Mapa projektu

sablona-csv/	
├─ NAVOD.md	# návod: nasadenie, NodePort, dôkaz perzistencie
├─ rozbor.md / rozbor.pdf	# tento dokument
├─ src/	
│ └─ index.php	# hlavná stránka + spracovanie CSV (bez štýlov)
├─ k8s/	# Kubernetes manifesty
│ └─ namespace.yaml	# 1. Namespace
│ └─ configmap.yaml	# 2. ConfigMap
│ └─ pv.yaml	# 3a. PersistentVolume
│ └─ pvc.yaml	# 3b. PersistentVolumeClaim
│ └─ deployment.yaml	# 4. InitContainer + Deployment
│ └─ service.yaml	# 5. Service NodePort
└─ k8s-notes/	# kópie manifestov s poznámkou na každom riadku

Pokrytie zadania (mapovanie na hodnotenie):

Požiadavka zo zadania	Súbor	Body
Namespace + štruktúra	namespace.yaml	5
ConfigMap s vplyvom na správanie	configmap.yaml	10
Funkčný InitContainer	deployment.yaml	10
PVC a zdieľaný filesystem	pv.yaml + pvc.yaml	10
Deployment a mountovanie volume	deployment.yaml	10
Service NodePort	service.yaml	10
Návod + popis spracovania	NAVOD.md	5

Čo aplikácia robí:

1. InitContainer pripraví na PVC priečinky input/ a output/ a (len pri prázdnom input/) nakopíruje sample CSV z ConfigMapu.
2. index.php pri každom requeste prejde input/*.csv; nové súbory spracuje — pridá hlavičku # PROCESSED: <timestamp> rows=N src=... a výsledok uloží do output/. Už spracované súbory nespracúva znova (idempotencia).
3. UI zobrazí tabuľky CSV + obsah output/.

1. k8s/namespace.yaml — Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: exam-horvath
```

Pole-po-poli

apiVersion: v1

- **Povinné? ÁNO.** Bez apiVersion kubectl manifest odmietne.
- Namespace je *core* objekt — API group je prázdna, takže len v1.

kind: Namespace

- Typ objektu: logické oddelenie zdrojov v klastri. Všetky namespaced objekty projektu (ConfigMap, PVC, Deployment, Service) žijú v ňom.

metadata.name: exam-horvath

- Dodržiava formát zo zadania exam-{meno}.
- Ostatné manifesty sa naň odkazujú cez metadata.namespace: exam-horvath.
- **Musí ísť kubectl apply ako prvý** — objekty v neexistujúcom namespace zlyhajú.

Čo je namespaced a čo nie

Namespaced (žije v NS)	Cluster-wide (mimo NS)
ConfigMap, PVC, Deployment, Pod, Service	PersistentVolume , Namespace, Node, StorageClass

2. k8s/configmap.yaml — ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: exam-horvath
data:
  APP_NAME: "Exam HorvathVK"
  LOG_LEVEL: "INFO"
  PROCESS_MODE: "prepend-marker"
  INPUT_DIR:  "/var/www/html/data/input"
  OUTPUT_DIR: "/var/www/html/data/output"
  SAMPLE_CSV_1: |
    name,age,city
    ...
  SAMPLE_CSV_2: |
    product,price,stock
    ...
```

Pole-po-poli

data: — kľúče a ich reálne použitie

Kľúč	Konzumuje	Povinný?	Efekt
APP_NAME	index.php cez getenv()	nie (fallback CSV Processor)	titulok stránky — viditeľný dôkaz vplyvu ConfigMapu
INPUT_DIR	index.php cez getenv()	nie (fallback rovnaká cesta)	odkiaľ sa čítajú CSV
OUTPUT_DIR	index.php cez getenv()	nie (fallback rovnaká cesta)	kam sa zapisujú výsledky
LOG_LEVEL, PROCESS_MODE	nikto	nie	len demonštračné
SAMPLE_CSV_1/2	volume cfg v Deploymente	ÁNO — chýbajúci kľúč = volume sa nenamontuje a pod nenašartuje	zdrojové dáta pre InitContainer

Dve cesty konzumácie toho istého ConfigMapu

1. **envFrom.configMapRef** (Deployment) — každý kľúč sa stane env premennou hlavného kontajnera. PHP ich číta cez getenv().
2. **Volume configMap s items:** — vybrané kľúče sa premietnu ako súbory /cfg/sample1.csv, /cfg/sample2.csv pre InitContainer.

YAML block scalar | (literal)

- Zachová nové riadky presne tak, ako sú napísané — ideálne na viacriadkový obsah súboru (CSV).
- **Pozor:** vnútri bloku nie je možné písať YAML komentáre — # by sa stal súčasťou dát.

Demo vplyvu ConfigMapu na správanie (na obhajobu)

```
kubectl -n exam-horvath patch cm app-config \
--type merge -p '{"data":{"APP_NAME":"Nove meno"}}'
```

```
kubectl -n exam-horvath rollout restart deployment/exam-app  
# refresh UI -> iný nadpis
```

(Restart je nutný — env premenné sa nastavujú len pri štarte kontajnera.)

3a. k8s/pv.yaml — PersistentVolume

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: exam-pv
spec:
  storageClassName: manual
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data/exam-horvath"
    type: DirectoryOrCreate
```

Pole-po-poli

metadata.name: exam-pv (bez namespace:)

- PV je **cluster-wide** objekt — nepatrí do žiadneho namespace. Preto ho `kubectl delete namespace` nezmaže (treba zvlášť `kubectl delete pv exam-pv`).

spec.storageClassName: manual

- „manual“ = **statické provisionovanie**: úložisko som pripravil ja, žiadny provisioner sa nezapája. PVC s rovnakou hodnotou sa naviaže práve sem.
- Iné šablóny vlastný PV nemajú — používajú defaultnú StorageClass minikube (standard), kde priečinok vytvára storage-provisioner addon automaticky.

spec.capacity.storage: 1Gi

- Koľko PV ponúka. Binding vyžaduje `PVC request <= PV capacity`.
- Pri `hostPath` sa kapacita **nevynucuje** — je to len deklarácia pre binding.

spec.accessModes: [ReadWriteOnce]

- RWO = čítanie aj zápis, ale len z **jedného node-u** naraz. Pri `replicas: 1` a jednom node (minikube) postačuje.
- Alternatívy: ROX (read-only z viacerých node-ov), RWX (zápis z viacerých — `hostPath` nepodporuje, treba NFS a pod.).

spec.hostPath.path + type: DirectoryOrCreate

- Dáta ležia priamo na disku node-u v `/mnt/data/exam-horvath` — prežijú zánik podu.
- `DirectoryOrCreate`: kubelet priečinok **vytvorí, ak neexistuje** — preto netreba žiadne ručné minikube `ssh -- mkdir`. Vytvorí ho ako `root:root`; vlastníctvo pre `www-data` následne rieši `InitContainer (chown -R 33:33)`.
- Bez `type: (default "")` sa existencia nekontroluje — chýbajúci priečinok síce zvyčajne vytvorí container runtime pri `bind-mounte`, ale nie je to garantované správanie. `DirectoryOrCreate` to robí deklaratívne.
- `hostPath` je vhodný **len na vývoj/minikube** — na produkcii by dáta boli priviazané na konkrétny node.

Implicitný default, ktorý tu nie je

- `persistentVolumeReclaimPolicy` — pre staticky vytvorené PV je default **Retain**: po zmazaní PVC dáta zostanú, PV prejde do stavu `Released` a na nový PVC sa nenaštie (drží starý `claim-Ref`) — treba PV vytvoriť znova.

3b. k8s/pvc.yaml — PersistentVolumeClaim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: exam-pvc
  namespace: exam-horvath
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Pole-po-poli

metadata.namespace: exam-horvath

- PVC **je** namespaced (na rozdiel od PV) — musí byť v namespace podu, ktorý ho mountuje.

spec.storageClassName: manual

- Musí sedieť s PV. Keby chýbal, zabrala by defaultná StorageClass a PVC by sa naviazal na dynamicky vytvorený volume namiesto nášho exam-pv.

spec.accessModes + spec.resources.requests.storage

- Požadovaný mód musí byť podmnožinou módov PV; požadovaná kapacita <= kapacity PV.
- PVC má len requests (žiadne limits).

Workflow PV <-> PVC binding

1. PV exam-pv existuje v stave Available.
2. PVC exam-pvc vznikne s požiadavkami (trieda manual, RWO, 1Gi).
3. K8s nájde najmenšie PV spĺňajúce **všetky** požiadavky -> Bound na oboch.
4. Pod referencuje PVC cez claimName — nikdy nie PV priamo.

Časté zlyhania

- PVC visí v Pending -> neexistuje PV s vyhovujúcou triedou/kapacitou/módom.
- PV v Released -> bol zmazaný starý PVC; nový sa nenaviaže, kým PV nevytvoríš znova (alebo neodstrániš claimRef).

4. k8s/deployment.yaml — InitContainer + Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: exam-app
  namespace: exam-horvath
spec:
  replicas: 1
  selector:
    matchLabels:
      app: exam-app
  template:
    metadata:
      labels:
        app: exam-app
    spec:
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: exam-pvc
        - name: cfg
          configMap:
            name: app-config
            items:
              - key: SAMPLE_CSV_1
                path: sample1.csv
              - key: SAMPLE_CSV_2
                path: sample2.csv
      initContainers:
        - name: init-data
          image: busybox:1.36
          command: ["sh", "-c"]
          args:
            - |
              set -e
              mkdir -p /data/input /data/output
              if [ -z "$(ls -A /data/input 2>/dev/null)" ]; then
                cp /cfg/sample1.csv /data/input/sample1.csv
                cp /cfg/sample2.csv /data/input/sample2.csv
              fi
              chown -R 33:33 /data
          volumeMounts:
            - name: data
              mountPath: /data
            - name: cfg
              mountPath: /cfg
      containers:
        - name: exam-app
          image: gitdockeracc/exam-app:latest
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: app-config
          volumeMounts:
```



```
- name: data
  mountPath: /var/www/html/data
```

Pole-po-poli

apiVersion: apps/v1

- Deployment nie je core objekt — patrí do API group apps.

spec.replicas: 1

- Jeden pod. Viac replík by s hostPath + RWO nedávalo zmysel (všetky by museli byť na tom istom node a zdieľali by ten istý priečinok).

spec.selector.matchLabels <-> template.metadata.labels

- **Musia sedieť**, inak kubectl apply zlyhá (selector does not match template labels).
- Cez label app: exam-app nachádza pody aj Service.

volumes: — dva zdroje dát

- data -> PVC exam-pvc -> PV exam-pv -> /mnt/data/exam-horvath na node. **Trvalé** úložisko zdieľané init aj hlavným kontajnerom.
- cfg -> ConfigMap app-config, items: vyberá len CSV kľúče a premietne ich ako súbory. **Read-only zdroj**, mení sa len cez kubectl apply ConfigMapu.

initContainers: — príprava dát

- Beží **pred** hlavným kontajnerom; pod štartuje až po jeho exit 0.
- busybox:1.36 — minimalistický image, stačí shell + mkdir/cp/chown.
- Skript po krokoch:
 1. set -e — pri prvej chybe skončí s nenulovým kódom (pod sa nespustí s polovičato pripravenými dátami).
 2. mkdir -p /data/input /data/output — idempotentné vytvorenie priečinkov.
 3. if [-z "\$(ls -A /data/input)"] — **kopíruj sample CSV len ak je input/ prázdny**. Toto je kľúč k dôkazu perzistencie: pri reštarte podu existujúce dáta nikdy neprepíše.
 4. chown -R 33:33 /data — UID/GID 33 = www-data v php:apache image; bez toho by Apache (bežiaci ako www-data) nemohol do PVC zapisovať. InitContainer beží ako **root**, preto chown môže urobiť.

containers: — hlavný kontajner

- image: gitdockeracc/exam-app:latest — php:8.2-apache + src/index.php.
- containerPort: 80 — informatívne pole (dokumentácia); skutočné otvorenie portu rieši proces Apache.
- envFrom.configMapRef — celý ConfigMap sa nasype ako env premenné.
- volumeMounts — **ten istý** volume data ako v initContaineri, mountnutý na /var/www/html/data (pod web rootom) -> zdieľaný filesystem medzi kontajnermi aj medzi reštartmi podu.

Poradie štartu podu

scheduler priradí pod na node

- > mount volumes (PVC musí byť Bound; DirectoryOrCreate vytvorí hostPath)
- > initContainer init-data (pripraví /data, exit 0)
- > hlavný kontajner exam-app (Apache na :80)

5. k8s/service.yaml — Service NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: exam-svc
  namespace: exam-horvath
spec:
  selector:
    app: exam-app
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Pole-po-poli

metadata.name: exam-svc

- Zároveň DNS meno v klastri: `exam-svc.exam-horvath.svc.cluster.local`.

spec.selector: app=exam-app

- Endpointy sa plnia automaticky podmi s týmto labelom. Ak selector neseďí, Service existuje, ale `kubectl get endpoints` je prázdny a spojenia visia.

spec.type: NodePort

- Otvorí port na **každom node** klastra -> prístup zvonku bez LoadBalancera.
- Zadanie explicitne žiada NodePort.

Tri porty a ich význam

Pole	Hodnota	Kto ho používa
port	80	iné pody v klastri (<code>exam-svc:80</code>)
targetPort	80	kam sa prevádzka pošle v kontajneri (Apache)
nodePort	30080	svet zvonku (<code>http://<node-ip>:30080</code>)

- nodePort musí byť v defaultnom rozsahu **30000-32767**; bez explicitnej hodnoty by K8s pridelil náhodný port z rozsahu.

6. src/index.php — aplikácia

Jediný PHP súbor, bez štýlov (čisté HTML + <table border="1">).

Logika spracovania

```
pre každý *.csv v INPUT_DIR (abecedne):
    ak output/<file> NEEXISTUJE:
        spočítaj dátové riadky (preskoč prázdne a #-riadky, -1 za header)
        hlavička = "# PROCESSED: <Y-m-d H:i:s> rows=N src=input/<file>"
        zapíš hlavička + pôvodný obsah do output/<file>
        status = "spracované teraz · <timestamp> · rows=N"
    inak:
        načítaj output/<file> (NEspracúvaj znova!)
        status = "už spracované · # PROCESSED: <pôvodný timestamp> ..."
    vyrenderuj tabuľku (riadky začínajúce # sa v zobrazení preskakujú)
nakoniec: výpis súborov v output/ s veľkosťami
```

Kľúčové vlastnosti

- **Idempotencia:** existencia output/<file> = súbor je spracovaný; ďalšie requesty aj nové pody ho už len čítajú. Pôvodný timestamp v hlavičke je preto **dôkazom perzistencie** (kap. 8).
- **Konfigurovateľnosť:** APP_NAME, INPUT_DIR, OUTPUT_DIR z env (ConfigMap), všetky s fallbackmi — appka beží aj bez ConfigMapu.
- **Bezpečný výstup:** všetko sa renderuje cez htmlspecialchars().
- mkdir() na začiatku je záchranná sieť — priečinky normálne pripraví InitContainer.

7. Prístup zvonku — NodePort bez obchádzania

7.1 Prečo `kubectl port-forward` NIE JE test NodePortu

`kubectl port-forward` tuneluje cez **API server priamo do podu** — obchádza kube-proxy, Service aj NodePort. Funguje aj keby Service neexistoval. Na demo NodePortu je preto nepoužiteľný.

7.2 Prečo nefunguje `localhost:30080` (docker driver)

S docker driverom beží „node“ ako kontajner s vlastnou IP (typicky 192.168.49.2). NodePort 30080 je otvorený na **tejto IP**, nie na hostiteľskom localhoste.

7.3 Varianty, ktoré idú skutočne cez NodePort

A) priamo na IP node-u — najčistejší dôkaz

```
curl http://$(minikube ip):30080/
```

B) minikube tunel KONČIACI na NodePorte

```
minikube service exam-svc -n exam-horvath --url
```

C) socat — stabilná localhost URL, prevádzka ide cez NodePort

```
sudo dnf install -y socat
```

```
socat TCP-LISTEN:3000,fork,reuseaddr TCP:$(minikube ip):30080 &
```

```
curl -I http://localhost:3000
```

Tok dát pri socat-e:

```
browser http://localhost:3000
```

```
-> socat (host :3000)
```

```
-> $(minikube ip):30080
```

```
<- reálny NodePort
```

```
-> Service exam-svc (selector app=exam-app)
```

```
-> Pod exam-app (Apache :80)
```

Správa socat-u: `jobs -l (zoznam), ss -tlnp | grep :3000 (počúva?), kill %1 (ukončenie).`

8. Dôkaz perzistencie

Princíp: prvé spracovanie CSV zapíše do output/ hlavičku # PROCESSED: <timestamp>. Ak po zabití podu vidíme **ten istý timestamp** so statusom „už spracované“, dáta prežili — nový pod nič neprepočítal, našiel hotový výsledok na PVC.

```
# 0. meno podu
POD=$(kubectl -n exam-horvath get pod -l app=exam-app \
-o jsonpath='{.items[0].metadata.name}')

# 1. vlastný testovací súbor (nemáme upload UI -> exec)
kubectl -n exam-horvath exec $POD -- sh -c \
'printf "city,temp\nNitra,21\nTrnava,19\n" > /var/www/html/data/input/test.csv'

# 2. refresh UI -> test.csv "spracované teraz . <timestamp>"; zapamätaj si čas
kubectl -n exam-horvath exec $POD -- head -1 /var/www/html/data/output/test.csv

# 3. zabi pod (Deployment vytvorí nový)
kubectl -n exam-horvath delete pods --all
kubectl -n exam-horvath get pods -w          # počkaj na Running 1/1

# 4. POZOR: nový pod = nové meno
POD=$(kubectl -n exam-horvath get pod -l app=exam-app \
-o jsonpath='{.items[0].metadata.name}')

# 5. dôkazy
kubectl -n exam-horvath exec $POD -- \
ls -la /var/www/html/data/input /var/www/html/data/output    # súbory existujú
kubectl -n exam-horvath exec $POD -- \
head -1 /var/www/html/data/output/test.csv                   # PÔVODNÝ timestamp
kubectl -n exam-horvath logs $POD -c init-data                # init nekopíroval
```

Interaktívne nazretie do podu:

```
kubectl -n exam-horvath exec -it $POD -- bash
ls -la /var/www/html/data/input/ /var/www/html/data/output/
head -1 /var/www/html/data/output/sample1.csv
exit
```

Nezávisle od podu — dáta priamo na node:

```
minikube ssh -- ls -la /mnt/data/exam-horvath/input /mnt/data/exam-horvath/output
minikube ssh -- head -1 /mnt/data/exam-horvath/output/test.csv
```

Čo prežije čo

Udalosť	Dáta prežijú?
reštart/zmazanie podu kubectl delete namespace	áno — PVC/PV žije nezávisle od podu dáta na disku áno; PV ostane Released (Retain) — treba ho vytvoriť znova
minikube stop && start (docker driver)	áno — kontajner node-u sa len zastaví
minikube stop && start (VM driver)	nie — /mnt/data nie je v perzistovaných cestách; použiť /data/...
minikube delete	nie — zmaže celý node aj disk

9. Príkazy — kompletný cheat sheet

9.1 Nasadenie

```
kubectl apply -f k8s/namespace.yaml      # namespace MUSÍ ísť prvý
kubectl apply -f k8s/                    # zvyšok (poradie v rámci apply -f rieši k8s)
```

Manuálne v logickom poradí: namespace -> configmap -> pv -> pvc -> deployment -> service.

9.2 Diagnostika

```
kubectl get all,cm,pvc -n exam-horvath    # prehľad objektov v NS
kubectl get pv exam-pv                    # PV je cluster-wide (bez -n); STATUS=Boun
kubectl get pods -n exam-horvath -w       # sleduj nábeh podu
kubectl describe pod <pod> -n exam-horvath # eventy: mount, image pull, init
kubectl logs -n exam-horvath -l app=exam-app -c init-data # čo spravil initContainer
kubectl logs -n exam-horvath -l app=exam-app # logy Apache
kubectl exec -it <pod> -n exam-horvath -- bash # shell v kontajneri
kubectl get endpoints exam-svc -n exam-horvath # Service vidí pod? (IP:80)
```

9.3 Testovanie

```
curl http://$(minikube ip):30080/        # cez NodePort
minikube service exam-svc -n exam-horvath --url # tunel na NodePort
socat TCP-LISTEN:3000,fork,reuseaddr TCP:$(minikube ip):30080 & # localhost:3000
```

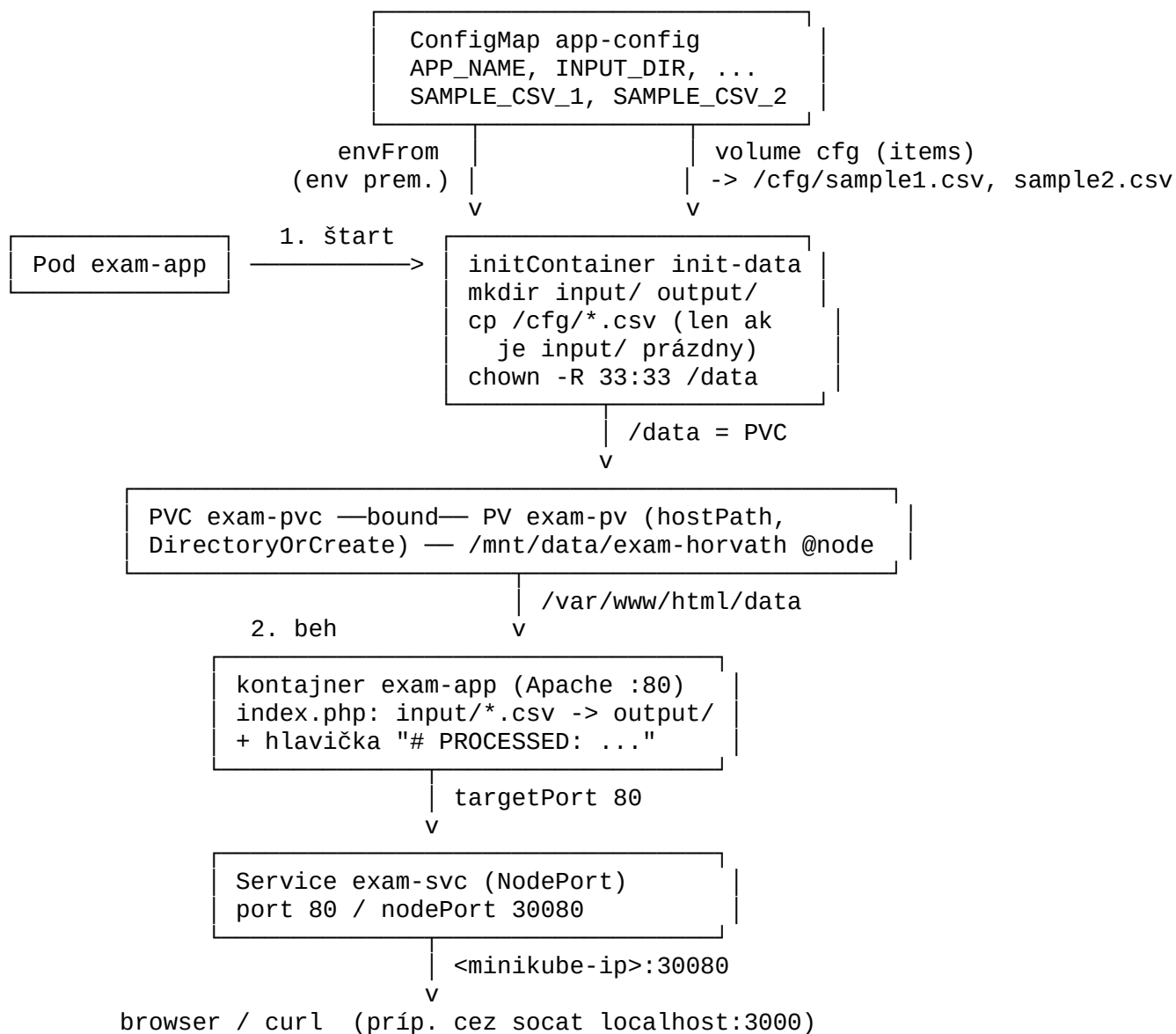
9.4 Update / re-deploy

```
kubectl rollout restart deployment/exam-app -n exam-horvath # nový pod (napr. po zmene C
kubectl rollout status deployment/exam-app -n exam-horvath # počkaj na dobehnutie
kubectl set image deployment/exam-app exam-app=gitdockeracc/exam-app:v2 -n exam-horvath
```

9.5 Cleanup

```
kubectl delete namespace exam-horvath    # zmaže všetko namespaced
kubectl delete pv exam-pv                 # PV treba zvlášť (cluster-wide)
minikube ssh -- "sudo rm -rf /mnt/data/exam-horvath" # voliteľne aj dáta
```

10. Tok dát — kompletný diagram



11. Checklist — pred odovzdaním

- ☐ kubectl get pods -n exam-horvath -> Running 1/1
- ☐ kubectl get pv -> exam-pv Bound
- ☐ kubectl logs ... -c init-data -> bez chýb
- ☐ UI beží cez http://\$(minikube ip):30080/ (NodePort, nie port-forward)
- ☐ sample CSV zobrazené ako tabuľky, output/ neprázdny
- ☐ demo ConfigMapu: zmena APP_NAME + rollout restart -> nový titulok
- ☐ demo perzistencie: delete pods --all -> rovnaký # PROCESSED: timestamp
- ☐ viem vysvetliť: prečo PV nie je namespaced, prečo chown 33:33, prečo DirectoryOrCreate, prečo port-forward obchádza NodePort